TrendPilot

AIRI Team PITB - AI Internship Task 2

End-to-End Agentic AI Assistant using a Local LLM (Ollama)

Submitted by: Hamza Nadeem

10	7.29 -> 8.21	15 -> 2	17
test cases	avg review score v1 -> v2	rule violations v1 -> v2	saved outputs (.md)

1. Problem Understanding and Project Idea

Students and junior engineers build good projects but struggle to present them: posts are generic, hooks are weak, hashtags are random and reel scripts run too long. At the same time, learners need structured study plans instead of random tutorials. The task asked for an agent - not a chatbot - that understands the request, plans, calls tools, reviews its own work and saves the result, running fully locally through Ollama.

From the suggested ideas I combined Option A (TrendPilot - content planner) and Option B (StudyBuddy). TrendPilot is the main agent: it turns any topic into a platform-ready content plan (idea, audience, hook, caption, hashtags, reel script, title, image prompt). StudyBuddy reuses the same agent engine with different tools (study planner, notes, quiz, progress tracking). One engine with two tool sets shows the main idea of agentic AI: the planner decides which tools a request needs.

Chatbot vs agent - what this project adds

Simple chatbot	TrendPilot agent
One prompt -> one answer	Understands the request, makes a plan and validates it
No tools	10 content tools + 5 StudyBuddy tools, chosen by an LLM planner
No quality control	Content Reviewer (LLM rubric + deterministic platform rules) and an automatic rewrite loop
Forgets everything	SQLite memory: past runs, learned preferences, study progress
Output only on screen	Markdown + JSON files saved automatically; CLI, Streamlit and FastAPI demos

2. Ollama Setup and Local Model

The model runs locally with Ollama on a Google Colab GPU, so no paid API is used. Installation: `curl -fsSL https://ollama.com/install.sh | sh`, `ollama serve` in the background, then `ollama pull gemma3:4b`. Python talks to the REST API (http://localhost:11434) with `requests`. Small fallback models (llama3.2:3b, qwen2.5:3b, phi3:mini) are used automatically if the main model is not available.

- Structured outputs: every tool sends a JSON schema in Ollama's `format` field, so a 4B model returns valid JSON; if parsing still fails the error is sent back to the model and it retries (max 2).
- temperature 0.7 for creative tools, 0.2-0.5 for reviewer/planner; num_ctx 4096; keep_alive keeps the model in GPU memory.
- The test prompt from the brief ('Give me 5 viral LinkedIn post ideas about AI internships.') was run - see the screenshot.

Google Colab - Ollama running gemma3:4b locally

```
$ ollama --version
ollama version is 0.40.14

$ ollama list
NAME         ID              SIZE      MODIFIED
gemma3:4b    68ceab7e468a    3.4 GB    Less than a second ago

$ ollama run gemma3:4b "Give me 5 viral LinkedIn post ideas about AI internships."
Okay, here are 5 viral LinkedIn post ideas about AI internships, designed to grab attention and encourage
engagement, with suggested visuals and a focus on what makes them shareable:

**1. Post Idea: The "AI Internship Myth" - Honest & Relatable**

* **Visual:** A slightly humorous graphic - maybe a bewildered-looking person surrounded by code, or a meme-
style image about feeling overwhelmed.
* **Text:** “Let’s be real, the AI internship landscape can feel *wild*.  I'm seeing a ton of hype about
‘guaranteed jobs’ and ‘instant mastery.’  The truth is, most internships are competitive, require serious
dedication, and a willingness to learn *a lot*.  What’s the #1 thing you wish you’d known going into your AI
internship?  Let’s destigmatize the learning process and share some real experiences. #AIInternship
#MachineLearning #DataScience #CareerAdvice #HonestTalk”
* **Why it’s shareable:**  Addresses a common frustration honestly.  Asking a direct question encourages
replies and starts a conversation. Uses relevant hashtags to broaden reach.
* **Engagement Goal:** High comment volume - people sharing their struggles or offering advice.


**2. Post Idea:  "Day in the Life" - Showcase a Real Internship**

* **Visual:** A carousel post with 3-5 images/short videos:
    *  Image 1: You at your desk working on a project.
    *  Image 2:  A screenshot of your code.
    *  Image 3:  A team meeting or collaboration.
    *  Image 4:  You learning something new (e.g., attending a webinar, reading documentation).
* **Text:** “Just another day in the life of an AI intern at [Company Name]!   I’m currently working on
[brief, intriguing project description – keep it accessible]. The best part?  I’m learning from incredible
mentors and building a portfolio I’m *proud* of.  Huge thanks to [mention a mentor or team]!  #AIInternship
#MachineLearningIntern #DataScienceIntern #WorkLife #CareerGoals #Innovation”
* **Why it’s shareable:**  Authentic, visually appealing, and provides a tangible glimpse into the experience.
  People love seeing what others are actually doing.
* **Engagement Goal:** Likes, comments (asking people what kind of projects they’d like to see), and
potentially shares if someone identifies with the experience.



**3. Post Idea: “Top 3 Skills for AI Internships” - Value-Driven**

* **Visual:** A visually appealing infographic-style graphic listing the 3 skills.  Use icons to represent
each skill.
* **Text:** “Want to land your dream AI internship?  Don't just focus on the fancy buzzwords! Here are the
*top 3* skills I’ve seen consistently sought after: 1 Strong Python Fundamentals 2 Basic Statistics &
Linear Algebra 3 Version Control (Git) What other skills do you think are crucial? Let's discuss!
#AIInternship #MachineLearningSkills #DataScienceSkills #SkillsDevelopment #CareerPrep”
* **Why it’s shareable:** Provides immediate, actionable advice.  Infographics are highly shareable.
* **Engagement Goal:**  Comments asking about other skills, sharing the post to help others.



**4. Post Idea:  “Ask Me Anything” – Leverage Your Experience**

* **Visual:** A professional headshot of you.
* **Text:** “I just wrapped up my AI internship at [Company Name], and I’m excited to share my experience!
I'm dedicating the next hour to answering your questions about the internship process, the projects I worked
on, the skills I developed, and anything else you’re curious about.  Drop your questions in the comments
below!  #AIInternship #AMA #AskMeAnything #MachineLearning #DataScience #CareerQuestions”
* **Why it’s shareable:**  Creates a sense of urgency and allows for direct interaction. People love asking
questions and getting personalized advice.
* **Engagement Goal:**  High volume of questions – you’ll need to be prepared to respond thoughtfully and
thoroughly.


... (32 more lines)
```

Screenshot 1 - Ollama running the local model (ollama --version, ollama list, test prompt).

3. Agent Workflow Design

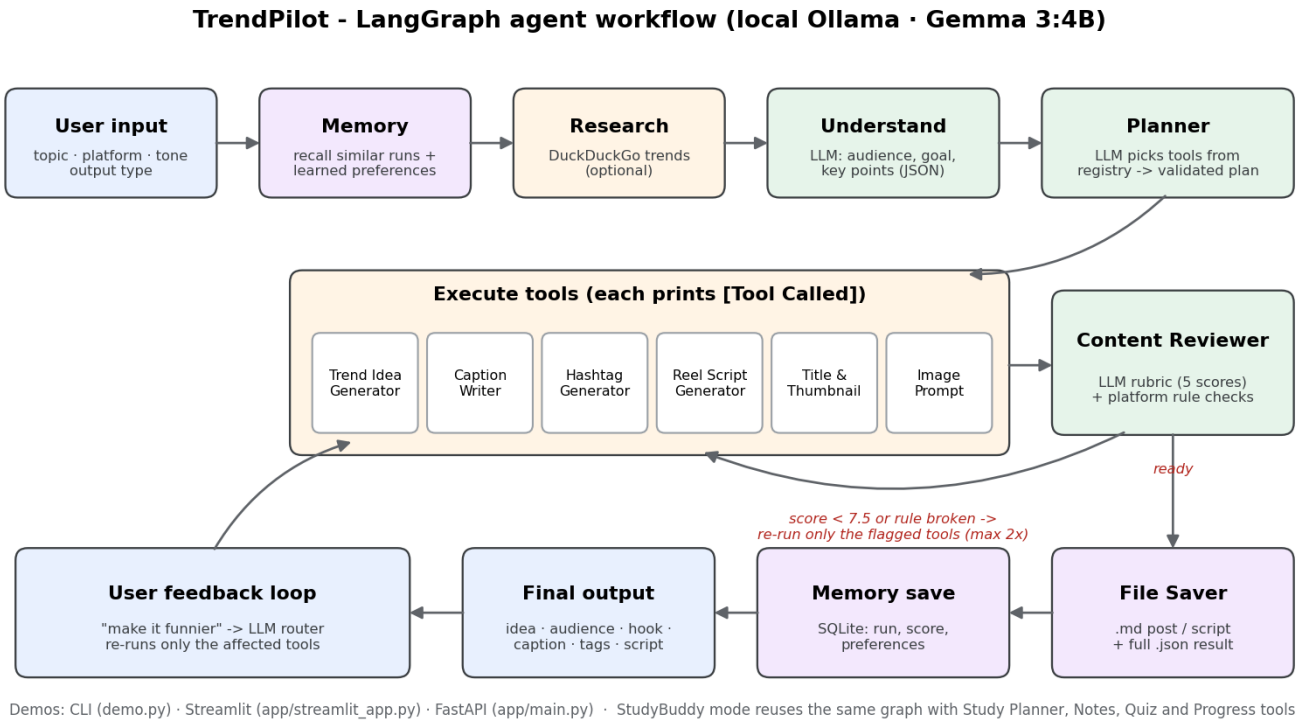


Figure - TrendPilot agent graph (LangGraph StateGraph).

- Step 1 - User input: topic, platform, tone, output type.
- Step 2 - Memory + research: recall similar past runs and learned preferences; optional DuckDuckGo trend search.
- Step 3 - Understand: the LLM extracts goal, target audience and key points as JSON.
- Step 4 - Plan: the LLM chooses tools from a registry; the plan is validated (unknown tools removed, missing required tools added, correct order).
- Step 5 - Execute: each tool runs and prints [Tool Called]: <name>.
- Step 6 - Review: Content Reviewer scores hook, clarity, platform fit, tone match and engagement (0-10) and runs platform rules (length, character limit, hashtag count, script duration, cliches, invented numbers).
- Step 7 - Refine loop: if the score is below 7.5 or a major rule is broken, only the flagged tools are re-run with the reviewer's suggestions (max 2 rounds).
- Step 8 - Save + final output: File Saver writes Markdown + JSON, memory stores the run, the final output is returned.
- User feedback loop: 'make the hook shorter' -> an LLM router picks the tools to re-run, and a _revised file is saved.

4. Tools Implemented

Tool	Mode	LLM	What it does
Simple Memory Tool	Both	No	Recall previous topics, platforms, styles and hooks from long-term memory (SQLite) so the new content is fresh.
Web Trend Research (DuckDuckGo)	TrendPilot	No	Search the web (DuckDuckGo) for fresh, real context about the topic so the content is current and factual.
Trend Idea Generator	TrendPilot	Yes	Generate 5 viral content angles for the topic and platform, score them and select the best one.

Tool	Mode	LLM	What it does
Caption Writer	TrendPilot	Yes	Write the hook, the full platform-ready post caption and a call to action in the requested tone.
Hashtag Generator	TrendPilot	Yes	Generate relevant hashtags for the platform (right count, broad + niche, no spam tags).
Reel Script Generator	TrendPilot	Yes	Create a 15-60 second short-video script: opening hook, explanation, visual suggestions and closing line.
Title and Thumbnail Text Generator	TrendPilot	Yes	Generate catchy video/post titles and short thumbnail text.
Post Image Prompt Generator	TrendPilot	Yes	Write a prompt for an AI image generator to create the post's cover image (optional extra).
Content Reviewer	TrendPilot	Yes	Review the content (hook strength, clarity, length, tone, hashtag relevance, platform readiness) and decide what to improve.
File Saver Tool	Both	No	Save the final result as Markdown + JSON in outputs/ (posts, scripts, full results). Mandatory last step.
Study Planner	StudyBuddy	Yes	Create a realistic day-by-day study plan with goals, tasks, time and resources.
Notes Generator	StudyBuddy	Yes	Write revision notes: summary, key concepts, cheat sheet and common mistakes.
Quiz Generator	StudyBuddy	Yes	Create multiple-choice quiz questions with answers and explanations.
Study Pack Reviewer	StudyBuddy	Yes	Review the plan, notes and quiz (coverage, pacing, clarity, quiz quality) and decide what to improve.
Progress Saver	StudyBuddy	No	Save the study plan and a day-by-day progress tracker to memory (SQLite) so progress can be updated later.

5. Memory and File Saving

- Short-term memory: the LangGraph state carries the request, plan, artifacts, review history and trace during a run.

- Long-term memory (SQLite, outputs/memory.db): every run with topic, platform, tone, hook and score; learned preferences (favourite platform/tone); study plans and day-by-day progress with quiz scores.

- Memory is used, not just stored: similar past topics are recalled (keyword similarity) and injected into the prompt so the agent avoids repeating the same hook.

- File saving: posts -> outputs/generated_posts/*.md, scripts -> outputs/generated_scripts/*.md, full results -> outputs/saved_results/*.json, study packs -> outputs/study_plans/. This run produced 17 Markdown and 15 JSON files.

- Every run is also logged to outputs/logs/agent_runs.jsonl (tools, score, time, LLM calls) for evaluation.

Saved agent outputs

```
$ ls outputs/*
outputs/generated_posts/  (10 files)
    .gitkeep
    ai_internship_experience_linkedin_20261104_074954_post.md
    data_annotation_struggles_linkedin_20261104_075144_post.md
    debugging_fastapi_x_twitter_20261104_075418_post.md
    final_year_project_linkedin_20261104_075302_post.md
    machine_learning_project_x_twitter_20261104_075112_post.md
    ... 4 more
outputs/generated_scripts/  (6 files)
    .gitkeep
    computer_vision_project_youtube_shor_20261104_075349_script.md
    model_failed_in_production_instagram_re_20261104_075227_script.md
    python_learning_journey_instagram_re_20261104_075039_script.md
    yolov8_helmet_detection_project_linkedin_20261104_073420_script.md
    yolov8_helmet_detection_project_linkedin_20261104_073445_revised_script.md
outputs/saved_results/  (14 files)
    .gitkeep
    ai_internship_experience_linkedin_20261104_074954.json
    computer_vision_project_youtube_shor_20261104_075349.json
    data_annotation_struggles_linkedin_20261104_075144.json
    debugging_fastapi_x_twitter_20261104_075418.json
    final_year_project_linkedin_20261104_075302.json
    ... 8 more
outputs/study_plans/  (2 files)
    .gitkeep
    study_object_detection_with_yolov8_5days_20261104_073558.md

$ cat outputs/generated_posts/yolov8_helmet_detection_project_linkedin_20261104_073420_post.md
# I want to create a viral LinkedIn post about my YOLOv8 helmet detection project

**Platform:** LinkedIn  |  **Tone:** Professional but exciting  |  **Output:** Post + Reel Script
**Generated:** 2026-11-04 07:34:20  |  **Model:** gemma3:4b  |  **Prompt version:** v2

## Content Idea
**The Data Dilemma** - I trained an AI to spot missing helmets. The hardest part wasn't the model - it was the
 data.

*Why it works:* Relatable — everyone struggles with data. Highlights a key issue in practical AI development.

## Target Audience
Computer vision engineers and robotics students interested in applying deep learning to safety applications.

## Viral Hook
> I trained an AI to spot missing helmets. The hardest part wasn't the model - it was the data.

## Post Caption
I trained an AI to spot missing helmets. The hardest part wasn't the model - it was the data.

The project utilizes the YOLOv8 model. The focus is on helmet detection, and the system operates in real-time.
  It demonstrates the application of AI in safety applications, built on core technology based on deep
learning.

Collecting sufficient, diverse data for helmet detection was a surprisingly significant challenge. We spent
considerable time ensuring the dataset represented various lighting conditions, head poses, and helmet styles.
  It’s easy to get lost in the technical details of model training, but a flawed dataset can completely derail
 your results.

This project highlights the importance of meticulous data preparation when applying deep learning — a critical
 element often overlooked.  We’re aiming to improve road safety by accurately identifying individuals without
helmets.

What are your biggest challenges when collecting data for computer vision projects? Share your thoughts below!
 

Let's discuss!

## Hashtags
#YOLOv8HelmetDetection #RealTimeObjectDetection #ComputerVisionSafety #DeepLearningRobotics

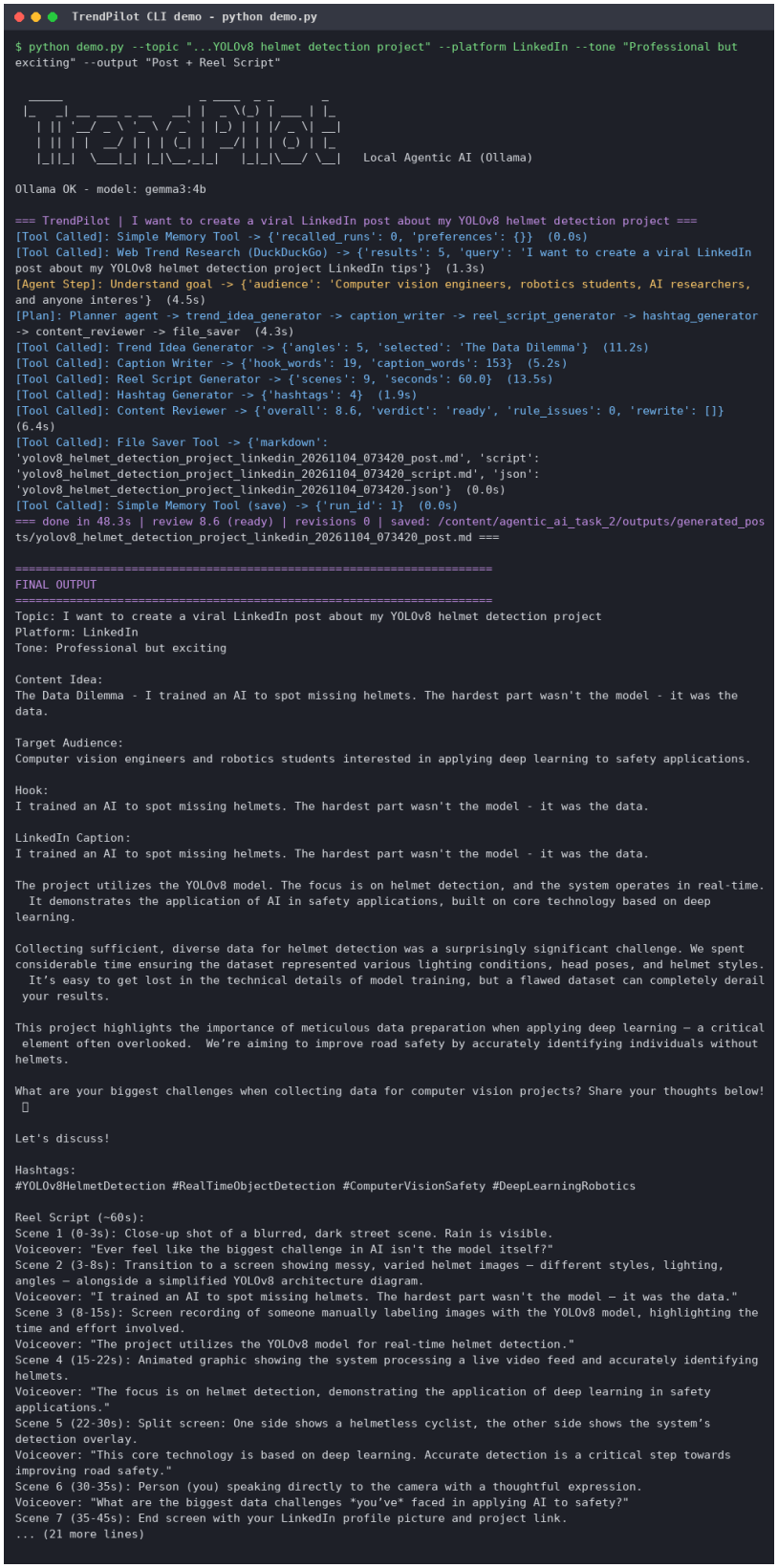
## Reel / Short Video Script (~60s)

| Time | Visual | Voiceover | On-screen text |
|---|---|---|---|
| 0-3s | Close-up shot of a blurred, dark street scene. Rain is visible. | Ever feel like the biggest
challenge in AI isn't the model itself? | The Data Dilemma |
| 3-8s | Transition to a screen showing messy, varied helmet images — different styles, lighting, angles —
alongside a simplified YOLOv8 architecture diagram. | I trained an AI to spot missing helmets. The hardest
part wasn't the model — it was the data. | Helmet Detection Project |
```

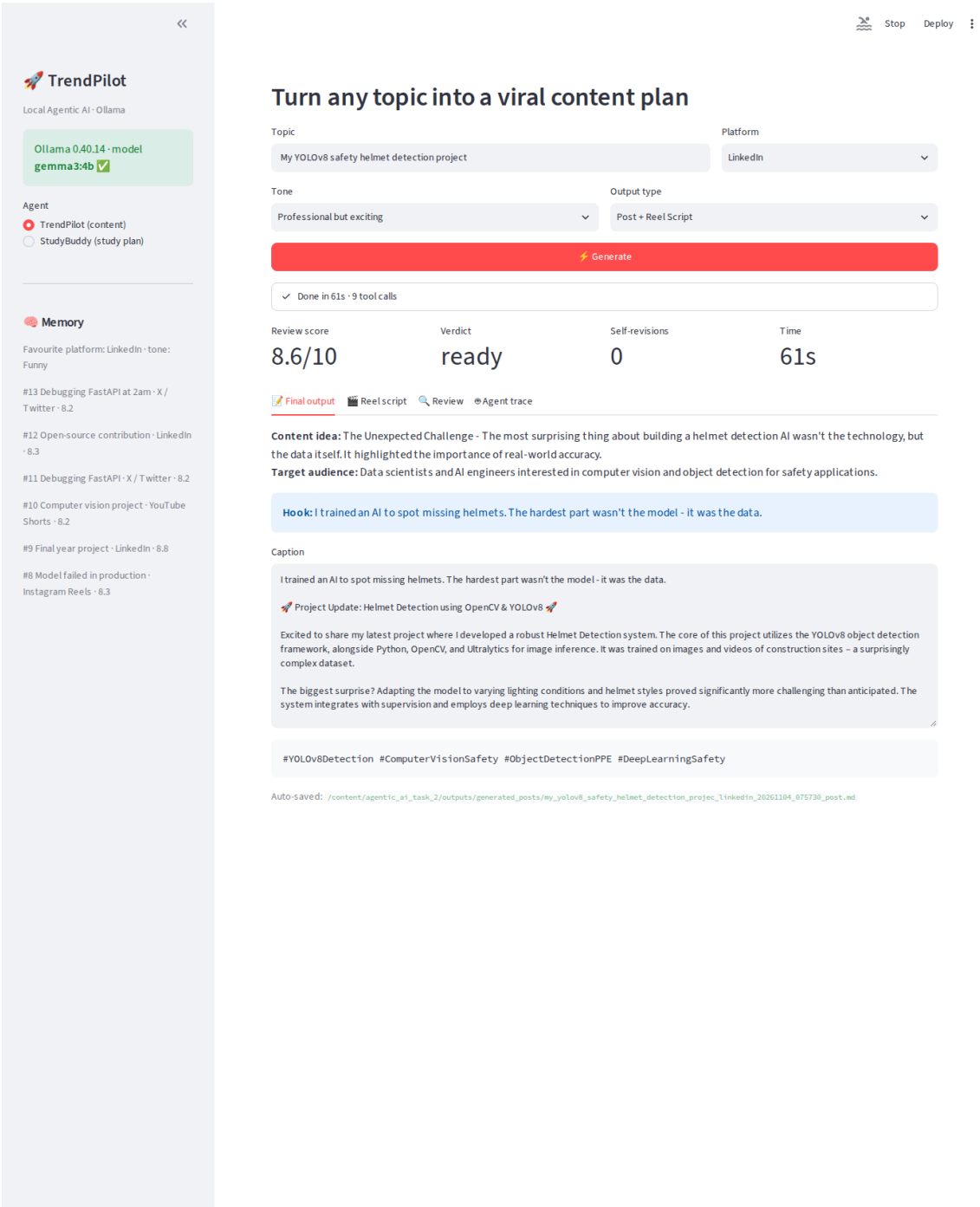
Screenshot 3 - saved output files and the content of a saved post.

6. Demo and Outputs

Three demos are included: CLI (python demo.py - interactive, with a feedback loop), Streamlit (app/streamlit_app.py - topic, platform, tone, output type, Generate, final output, Save result, live agent trace, review chart, StudyBuddy tab) and FastAPI (app/main.py - POST /generate-content, /improve-content, /study-plan, /study-progress, GET /history).



Screenshot 2 - CLI demo: [Tool Called] logs and the final output.



Screenshot - Streamlit demo after one generation.

Example output (main example from the brief)

Input: I want to create a viral LinkedIn post about my YOLOv8 helmet detection project | LinkedIn | Professional but exciting | Post + Reel Script

Tools used: Simple Memory Tool -> Web Trend Research (DuckDuckGo) -> Trend Idea Generator -> Caption Writer -> Reel Script Generator -> Hashtag Generator -> Content Reviewer

Field	Agent output
Content idea	The Data Dilemma - I trained an AI to spot missing helmets. The hardest part wasn't the model - it was the data.
Target audience	Computer vision engineers and robotics students interested in applying deep learning to safety applications.
Hook	I trained an AI to spot missing helmets. The hardest part wasn't the model - it was the data.

Field	Agent output
Caption	I trained an AI to spot missing helmets. The hardest part wasn't the model - it was the data. The project utilizes the YOLOv8 model. The focus is on helmet detection, and the system operates in real-time. It demonstrates the application of AI in safety applications, built on core technology based on deep learning. Collecting sufficient, diverse data for helmet detection was a surprisingly significant challenge. We spent considerable time ensuring the dataset represented various lighting conditions, head poses, and helmet styles. It's easy to get lost in the technical details of model training, but a flawed dataset can completely derail your results. This project highlights the importance of meticulous data preparation when applying deep learning – a critical element often overlooked. We're aiming to improve road safety by accurately identifying individuals without helmets. What are your biggest challenges when collecting data for computer vision projects? Share your thoughts below! Let's discuss!
Hashtags	#YOLOv8HelmetDetection #RealTimeObjectDetection #ComputerVisionSafety #DeepLearningRobotics
Reel script	0-3s: [Close-up shot of a blurred, dark street scene. Rain is visible.] Ever feel like the biggest challenge in AI isn't the model itself? 3-8s: [Transition to a screen showing messy, varied helmet images – different styles, lighting, angles – alongside a simplified YOLOv8 architecture diagram.] I trained an AI to spot missing helmets. The hardest part wasn't the model – it was the data. 8-15s: [Screen recording of someone manually labeling images with the YOLOv8 model, highlighting the time and effort involved.] The project utilizes the YOLOv8 model for real-time helmet detection. 15-22s: [Animated graphic showing the system processing a live video feed and accurately identifying helmets.] The focus is on helmet detection, demonstrating the application of deep learning in safety applications. 22-30s: [Split screen: One side shows a helmetless cyclist, the other side shows the system's detection overlay.] This core technology is based on deep learning. Accurate detection is a critical step towards improving road safety. 30-35s: [Person (you) speaking directly to the camera with a thoughtful expression.] What are the biggest data challenges *you've* faced in applying AI to safety? 35-45s: [End screen with your LinkedIn profile picture and project link.] Explore the full project details here: [link] 45-50s: [Quick montage of the images and visualizations from the video.] 50-60s: [Final shot – a cyclist wearing a helmet riding confidently on a sunny road.] Let's build safer roads, together.
Title / thumbnail	/
Review	8.6/10 - Streamline the reel script to highlight the core challenge – data – more powerfully. Reduce the screen recordings to focus on the key visual impact.; Consider a more evocative opening for the reel. Instead of stating the problem, show a brief, impactful visual of a near-miss situation (without explicitly showing a helmetless rider) to immediately grab attention.
Saved file	/content/agentic_ai_task_2/outputs/generated_posts/yolov8_helmet_detection_project_linkedin_20261104_073420_post.md

User feedback loop

Feedback: "Make the hook shorter and more surprising, and add one line about what I learned from the project." -> tools re-run: caption_writer, reel_script_generator. New hook: I trained an AI to spot missing helmets. The hardest part wasn't the model - it was the data.

StudyBuddy mode

Request: Object detection with YOLOv8 - 5 days, Beginner. Output: 'YOLOv8 Object Detection: Your 5-Day Launchpad' with 5 daily plans, 10 key concepts and 8 quiz questions; progress is tracked in SQLite.

Day	Focus	Tasks
1	Introduction to Object Detection and YOLO	Read: PyTorch documentation on Automatic Differentiation - https://pytorch.org/docs/stable/autograd.html; Watch: YouTube - 'YOLOv8 Explained' - (approx. 30 mins) - https://www.youtube.com/watch?v=I4qgH-4wVj0; Read: YOLOv8 Documentation - Overview and Architecture - https://docs.ultralytics.com/yolov8/getting_started/introduction
2	YOLOv8 Training and Configuration	Read: YOLOv8 Documentation - Training - https://docs.ultralytics.com/yolov8/training/; Watch: YouTube - 'YOLOv8 Training Tutorial' - (approx. 45 mins) - https://www.youtube.com/watch?v=9nQ-wV33J5I; Code: Experiment with a pre-trained YOLOv8 model's configuration file - (approx. 30 mins)
3	Inference with YOLOv8	Read: YOLOv8 Documentation - Inference - https://docs.ultralytics.com/yolov8/inference/; Code: Run a pre-trained YOLOv8 model on a sample image - (approx. 60 mins); Code: Experiment with different inference parameters (e.g., confidence threshold) - (approx. 30 mins)
4	Custom Dataset and Training	Read: YOLOv8 Documentation - Custom Dataset - https://docs.ultralytics.com/yolov8/data/custom_dataset; Code: Annotate a small custom dataset using a tool like LabelImg - (approx. 60 mins); Code: Train a YOLOv8 model on your custom dataset - (approx. 60 mins)
5	Project: Simple Object Detection Application	Choose a simple object detection task (e.g., detecting cars in images).; Code: Integrate a trained YOLOv8 model into a basic application (e.g., using Python and OpenCV) - (approx. 90 mins); Code: Implement basic user interface to display detected objects - (approx. 30 mins)

7. Testing (10 test cases)

All 10 inputs from the brief were run twice: v1 = baseline (simple prompts, fixed pipeline, no review loop) and v2 = final agent (improved prompts, LLM planner, reviewer loop). Average review score went from 7.29 to 8.21, rule violations from 15 to 2, and useful outputs from 3/10 to 9/10. In v2 the reviewer loop automatically rewrote 0 of 10 cases. Average time per case: 36.5 s.

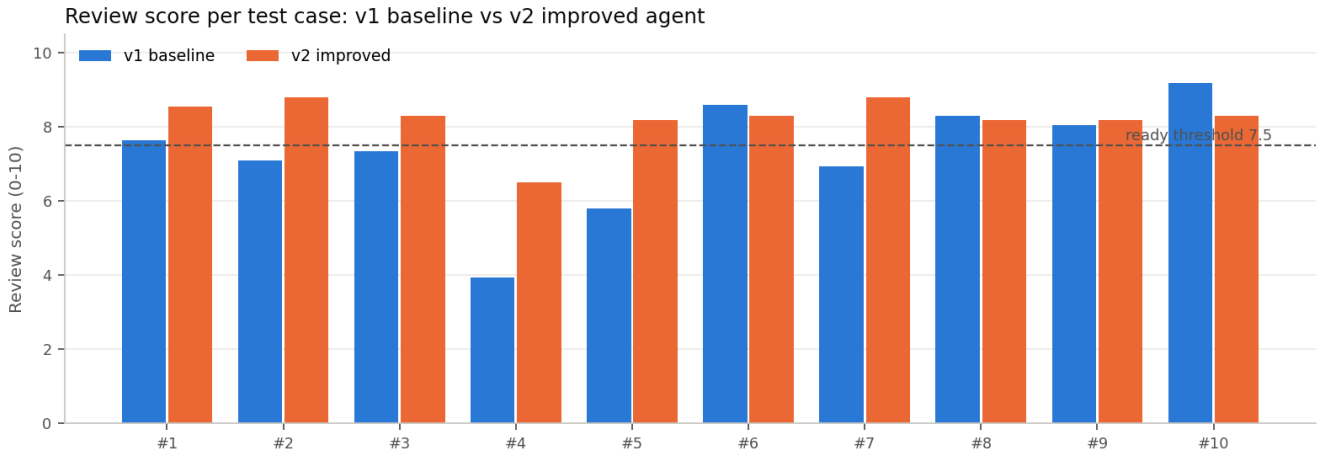


Figure - review score per test case (dashed line = ready threshold).

#	Input	Tools used	Final output (preview)	Score	Useful
1	YOLOv8 helmet detection | LinkedIn | Professional | Post + hashtags	Web Trend Research (DuckDuckGo), Trend Idea, Caption Writer, Hashtag, Content Reviewer, File Saver	I trained an AI to spot missing helmets. The hardest part wasn't the model - it was the data.	8.55	Yes
2	AI internship experience | LinkedIn | Motivational | Post	Web Trend Research (DuckDuckGo), Trend Idea, Caption Writer, Hashtag, Content Reviewer, File Saver	Seriously, keep that LinkedIn profile updated – it's your first impression.	8.8	Yes
3	Python learning journey | Instagram Reels | Funny | Reel script	Web Trend Research (DuckDuckGo), Trend Idea, Reel Script, Hashtag, Title and Thumbnail Text, Content Reviewer, File Saver	Stop scrolling! Seriously. Let's code.	8.3	Yes
4	Machine learning project | X / Twitter | Short and catchy | Short post	Web Trend Research (DuckDuckGo), Trend Idea, Caption Writer, Hashtag, Content Reviewer, File Saver	Forget labeled datasets. Let's build a massive X (Twitter) one.	6.5	Partly
5	Data annotation struggles | LinkedIn | Humorous | Post	Web Trend Research (DuckDuckGo), Trend Idea, Caption Writer, Hashtag, Content Reviewer, File Saver	That starter assessment... it's like a digital gatekeeper.	8.2	Yes
6	Model failed in production | Instagram Reels | Meme style | Reel script	Web Trend Research (DuckDuckGo), Trend Idea, Reel Script, Hashtag, Title and Thumbnail Text, Content Reviewer, File Saver	Your Reel just died? Don't panic.	8.3	Yes
7	Final year project | LinkedIn | Professional | Portfolio post	Web Trend Research (DuckDuckGo), Trend Idea, Caption Writer, Hashtag, Content Reviewer, File Saver	I trained an AI to spot missing helmets. The hardest part wasn't the model.	8.8	Yes
8	Computer vision project | YouTube Shorts | Exciting | Video script	Web Trend Research (DuckDuckGo), Trend Idea, Reel Script, Hashtag, Title and Thumbnail Text, Content Reviewer, File Saver	My AI fails... spectacularly. Building a computer vision project isn't always glamorous.	8.2	Yes

#	Input	Tools used	Final output (preview)	Score	Useful
9	Debugging FastAPI | X / Twitter | Funny | Short post	Web Trend Research (DuckDuckGo), Trend Idea, Caption Writer, Hashtag, Content Reviewer, File Saver	FastAPI debugging feels like wrestling a Python dragon.	8.2	Yes
10	Open-source contribution | LinkedIn | Inspirational | Post	Web Trend Research (DuckDuckGo), Trend Idea, Caption Writer, Hashtag, Content Reviewer, File Saver	My resume looked like a chaotic list of projects – it needed a serious upgrade.	8.3	Yes

v1 baseline vs v2 improved

#	Topic	Platform	v1 score	v1 issues	v2 score	v2 issues	Self-revisions
1	YOLOv8 helmet detection	LinkedIn	7.65	2	8.55	1	0
2	AI internship experience	LinkedIn	7.1	2	8.8	0	0
3	Python learning journey	Instagram Reels	7.35	2	8.3	0	0
4	Machine learning project	X / Twitter	3.95	3	6.5	1	0
5	Data annotation struggles	LinkedIn	5.8	2	8.2	0	0
6	Model failed in production	Instagram Reels	8.6	0	8.3	0	0
7	Final year project	LinkedIn	6.95	3	8.8	0	0
8	Computer vision project	YouTube Shorts	8.3	0	8.2	0	0
9	Debugging FastAPI	X / Twitter	8.05	1	8.2	0	0
10	Open-source contribution	LinkedIn	9.2	0	8.3	0	0

Strong outputs: #1 YOLOv8 helmet detection (8.55); #2 AI internship experience (8.8); #3 Python learning journey (8.3); #5 Data annotation struggles (8.2); #6 Model failed in production (8.3); #7 Final year project (8.8).

Weaker outputs: #4 Machine learning project (6.5) - low reviewer score.

8. Error Analysis

Test case	Problem found	Possible reason	Fix applied	Result after fix
#4 Machine learning project (X / Twitter, Short and catchy)	350 characters incl. hashtags > X / Twitter limit 280.	X/Twitter's 280-character limit (incl. hashtags) was never mentioned	Added the 280-character rule and an automatic character-count check	fixed - score 3.95 -> 6.5
#5 Data annotation struggles (LinkedIn, Humorous)	Caption has only 50 words; LinkedIn target is 90-260.	No hashtag count was given for the platform	Platform-specific hashtag counts + automatic cap in the hashtag tool	fixed - score 5.8 -> 8.2
#7 Final year project (LinkedIn, Professional)	Hashtags are mixed into the caption text.	No hashtag count was given for the platform	Platform-specific hashtag counts + automatic cap in the hashtag tool	fixed - score 6.95 -> 8.8
#2 AI internship experience (LinkedIn, Motivational)	Hashtags are mixed into the caption text.	No hashtag count was given for the platform	Platform-specific hashtag counts + automatic cap in the hashtag tool	fixed - score 7.1 -> 8.8
#3 Python learning journey (Instagram Reels, Funny)	15 hashtags; Instagram Reels works best with 5-10.	Model returned broken/invalid JSON for a tool	Ollama structured outputs (JSON schema) + automatic retry with the parse error	fixed - score 7.35 -> 8.3
#1 YOLOv8 helmet detection (LinkedIn, Professional)	14 hashtags; LinkedIn works best with 3-5.	Model returned broken/invalid JSON for a tool	Ollama structured outputs (JSON schema) + automatic retry with the parse error	fixed - score 7.65 -> 8.55
#9 Debugging FastAPI (X / Twitter, Funny)	3 hashtags; X / Twitter works best with 1-2.	No hashtag count was given for the platform	Platform-specific hashtag counts + automatic cap in the hashtag tool	fixed - score 8.05 -> 8.2

9. What I Learned

- How to run and serve a local LLM with Ollama on a Colab GPU and call it from Python through its REST API.
- The difference between a chatbot and an agent: planning, tool selection, state, self-review and memory.
- How to force a small model to return reliable JSON (schemas, validation and retry with the error message).
- That small models need specific prompts: audience, platform rules, few-shot examples and length limits changed the results a lot.
- How to combine an LLM judge with deterministic rule checks so the agent can find and fix its own mistakes.
- How to build LangGraph state machines with conditional edges (review -> refine loop).
- How to test an agent systematically with fixed test cases, baseline vs improved versions and an error-analysis table.

10. Future Improvements

- Use a larger local model (e.g. Gemma 3 12B or Qwen 2.5 7B) for funnier, more natural humour and meme-style content.
- Add a human-in-the-loop step in the UI: show 3 hooks and let the user pick before the caption is written.
- Store platform analytics (likes/comments) back into memory so the agent learns which hooks perform best.
- Add a fact-checking tool that compares claims in the caption with the user's README / project files (RAG).
- Add image generation for the cover image prompt (e.g. Stable Diffusion locally) and export the post as PDF/carousel.
- Schedule posts and track a content calendar in SQLite; deploy the FastAPI backend with Docker.

11. Repository Structure

```
agentic_ai_task_2/
  app/ main.py (FastAPI) agent.py (LangGraph) tools.py memory.py prompts.py llm.py quality.py
       evaluation.py config.py streamlit_app.py
  outputs/ generated_posts/ generated_scripts/ saved_results/ study_plans/ logs/
  tests/ test_cases.md   screenshots/   report/ final_report.pdf linkedin_portfolio_report.md
  notebooks/ TrendPilot_Colab.ipynb   scripts/   README.md   requirements.txt   demo.py
```